Progetto 1: Server HTTP Concorrente

Gestione Prenotazione Aule Studio in C

Massimo Mantineo (541924) Pierluca Tino Castorina (545101)

Università degli Studi di Messina
Dipartimento MIFT - Laurea in Informatica

Anno Accademico 2024/2025

1. Problem Statement: Obiettivi del Progetto

Il Problema

Implementare un server per la gestione di prenotazioni in grado di servire simultaneamente molteplici client, garantendo la coerenza dei dati e riducendo l'overhead computazionale di sistema.

Le Soluzioni Adottate:

- **C Nativo & POSIX:** Utilizzo delle *system call* di basso livello anziché di framework moderni (Node.js, Spring) per massimizzare il controllo delle risorse.
- **Architettura a Thread Pool:** Sostituzione del pattern classico basato su `fork()` con un pool di worker thread pre-allocati.
- **Sincronizzazione Esplicita:** Impiego combinato di Semafori (per la comunicazione di rete) e Mutex (per l'accesso in mutua esclusione al database in memoria).

2. Stato dell'Arte: Modelli di Concorrenza

L'evoluzione della gestione dei socket ha portato a diversi modelli architetturali:

- **Processo per Connessione (`fork()`)**: Semplice ma inefficiente (alto overhead di memoria e context switching).
- **Thread per Connessione**: Più leggero ma soffre del problema *C10k* (esaurimento stack dei thread).
- **Event-Driven (`epoll/select`)**: Estremamente performante ma complesso da implementare (I/O asincrono).
- **Thread Pool (Nostra Scelta)**: Il giusto compromesso tra prestazioni e semplicità gestendo un numero fisso di thread worker pronti all'uso.

Framework Moderni

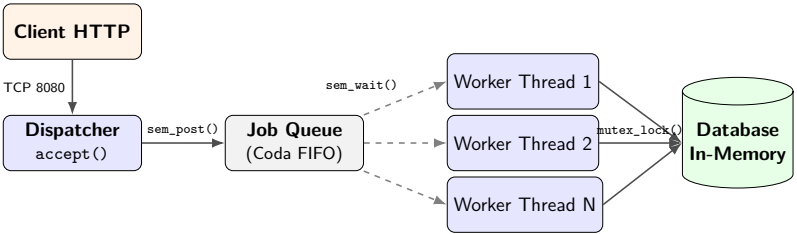
Linguaggi come Go o Node.js automatizzano questa gestione tramite *Goroutines* o *Event Loops*, ma il C/POSIX permette una comprensione profonda della gestione delle risorse del kernel.

3. Metodologia: Lo Stack Tecnologico

Il progetto è stato sviluppato e validato utilizzando esclusivamente strumenti standard e open-source:

- **Linguaggio**: C99 con estensioni POSIX.
- **Build System**: `Makefile` per l'automazione della compilazione.
- **Debugging**: `GDB` e `Valgrind` (per l'analisi dei leak di memoria).
- **Networking**: `Wireshark` e `tcpdump` per l'analisi dei pacchetti HTTP/TCP.
- **Benchmarking**: `Apache Bench (ab)` per lo stress test concorrente.
- **Version Control**: `Git` per la gestione del codice sorgente.

4. Metodologia: Architettura a Thread Pool



5. Metodologia: Handshake TCP

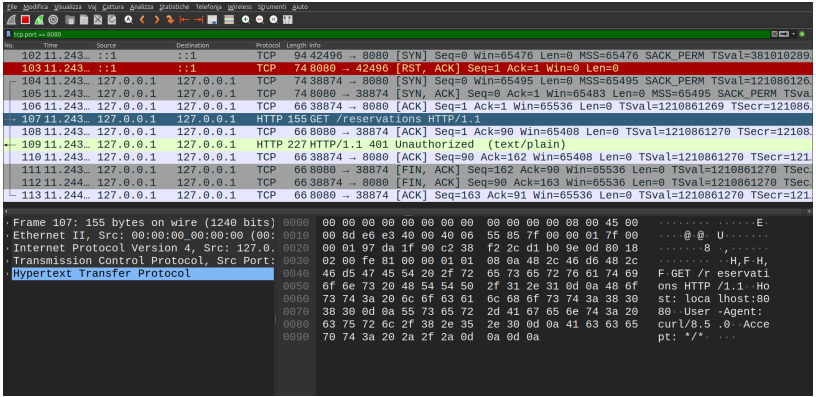


Figura: Tracciato Wireshark del Three-Way Handshake TCP.

Autenticazione

Le rotte sensibili richiedono l'header `Authorization: Basic` (token Base64).

6. Metodologia: Sincronizzazione e Flusso

L'utilizzo dei **Semafori** (`sem_t`) permette di gestire la comunicazione tra thread evitando il *busy-waiting* (polling attivo).

- **Gestione degli Stati**: Il sistema operativo pone i thread worker in *Blocked State* quando la coda è vuota, azzerando il consumo di CPU.
- **Segnalazione (Signal/Wait)**:
  - `sem_wait()`: Decrementa il contatore; se è zero, il thread dorme.
  - `sem_post()`: Incrementa il contatore e risveglia un thread dormiente.
- **Efficienza**: Rispetto ad un approccio basato su `fork()`, il pool di thread riduce drasticamente l'overhead di *Context Switching*.

7. Implementazione: Strutture Dati

L'architettura poggia su due strutture fondamentali per la gestione dello stato e della sincronizzazione.

```
typedef struct job {
    void (*function)(void*); // Funzione da eseguire
    void* arg;               // Argomento (es. socket descriptor)
    struct job* next;        // Puntatore per coda FIFO
} job_t;

typedef struct {
    pthread_t* threads;        // Array di worker thread
    job_t* queue_head;         // Testa della coda (estrazione)
    job_t* queue_tail;         // Coda della lista (inserimento)
    pthread_mutex_t queue_mutex; // Protezione della coda
    sem_t queue_sem;           // Conteggio task pronti
    int shutdown;              // Flag di terminazione controllata
} thread_pool_t;
```

8. Implementazione: Inizializzazione Pool

All'avvio, il server alloca le risorse di sincronizzazione e lancia i thread worker.

```
thread_pool_t* thread_pool_init(int num_threads) {
    thread_pool_t* pool = malloc(sizeof(thread_pool_t));

    // Inizializzazione primitiva di sincronizzazione
    pthread_mutex_init(&pool->queue_mutex, NULL);
    sem_init(&pool->queue_sem, 0, 0); // Semaforo parte da 0

    pool->threads = malloc(num_threads * sizeof(pthread_t));
    for (int i = 0; i < num_threads; i++) {
        // Ogni thread esegue la funzione 'thread_worker'
        pthread_create(&pool->threads[i], NULL, thread_worker, (void*)pool);
    }
    return pool;
}
```

9. Implementazione: Pattern Produttore-Consumatore

Dispatcher (Produttore)

```
// Accettazione e creazione job
int* client_sock = malloc(sizeof(int));
*client_sock = accept(...);

pthread_mutex_lock(&pool->queue_mutex);
// Aggiunta in coda (FIFO)
pool->queue_tail->next = job;
pool->queue_tail = job;
pthread_mutex_unlock(&pool->queue_mutex);

// Sveglia un thread worker
sem_post(&pool->queue_sem);
```

Worker (Consumatore)

```
while (1) {
    // Attesa passiva (nessun busy-waiting)
    sem_wait(&pool->queue_sem);

    pthread_mutex_lock(&pool->queue_mutex);
    if (pool->shutdown) {
        pthread_mutex_unlock(&pool->
            queue_mutex);
        break; // Shutdown controllato
    }
    // Estrazione job dalla coda...
    pthread_mutex_unlock(&pool->queue_mutex);

    handle_client(job);
}
```

10. Implementazione: Shutdown Sicuro

La chiusura del server richiede un coordinamento per evitare *deadlock* o thread rimasti in attesa sul semaforo.

```
void thread_pool_destroy(thread_pool_t* pool) {
    pthread_mutex_lock(&pool->queue_mutex);
    pool->shutdown = 1; // Segnale di stop
    pthread_mutex_unlock(&pool->queue_mutex);

    // Svegliamo tutti i thread in attesa su sem_wait()
    for (int i = 0; i < pool->num_threads; i++) {
        sem_post(&pool->queue_sem);
    }

    // Join per attendere la terminazione effettiva
    for (int i = 0; i < pool->num_threads; i++) {
        pthread_join(pool->threads[i], NULL);
    }
}
```

11. Metodologia: Mutual Exclusion

Il **Mutex** (*Mutual Exclusion*) garantisce che una determinata porzione di codice, detta **Sezione Critica**, sia eseguita da un solo thread alla volta.

- **Prevenzione Data Race**: Impedisce a più thread di modificare contemporaneamente la stessa variabile (es. il contatore delle prenotazioni).
- **Atomicità**: Le operazioni sul database in-memory (scrittura, aggiornamento, shift) devono apparire come singole unità indivisibili.
- **Isolamento**: Ogni worker thread ha una visione coerente della memoria mentre detiene il lock.

Regola d'oro

La durata del lock deve essere ridotta al minimo per non serializzare l'esecuzione del server.

12. Implementazione: Thread Safety

L'accesso alla risorsa condivisa (il vettore delle prenotazioni) è protetto da un Mutex per garantire la coerenza dei dati durante operazioni concorrenti.

```
int rm_create(resource_manager_t* rm, const char* room, ...) {
    pthread_mutex_lock(&rm->mutex); // ACQUISIZIONE LOCK

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // Scrittura sicura nell'array condiviso
    strncpy(rm->reservations[rm->count].room, room, 127);
    rm->count++;

    pthread_mutex_unlock(&rm->mutex); // RILASCIO LOCK
    return rm->count - 1;
}
```

13. Implementazione: Eliminazione Atomica

Le operazioni logiche complesse, come la rimozione con ricompattazione (shift), devono avvenire integralmente sotto lock.

```
int rm_delete(resource_manager_t* rm, int id) {
    pthread_mutex_lock(&rm->mutex); // Sezione Critica
    for (int i = 0; i < rm->count; i++) {
        if (rm->reservations[i].id == id) {
            // Shift a sinistra per ricompattare l'array
            for (int j = i; j < rm->count - 1; j++) {
                rm->reservations[j] = rm->reservations[j + 1];
            }
            rm->count--;
            pthread_mutex_unlock(&rm->mutex);
            return 0;
        }
    }
    pthread_mutex_unlock(&rm->mutex);
    return -1;
}
```

14. Implementazione: Analisi del Protocollo (Parsing)

Il server deve interpretare i byte grezzi trasformandoli in una struttura logica (*Request*).

```
int parse_http_request(const char* raw, http_request_t* req) {
    // 1. Ricerca fine della riga (\r\n)
    const char* line_end = strstr(raw, "\r\n");
    // 2. Estrazione Metodo e URI
    sscanf(raw, "%s␣%s", req->method, req->uri);
    // 3. Individuazione del Body (dopo \r\n\r\n)
    const char* body_start = strstr(raw, "\r\n\r\n");
    if (body_start) {
        strncpy(req->body, body_start + 4, MAX_BODY_LEN - 1);
    }
    return 0;
}
```

Nota: L'uso di strstr e sscanf garantisce il rispetto dello standard RFC 7230.

15. Implementazione: Specifiche API REST

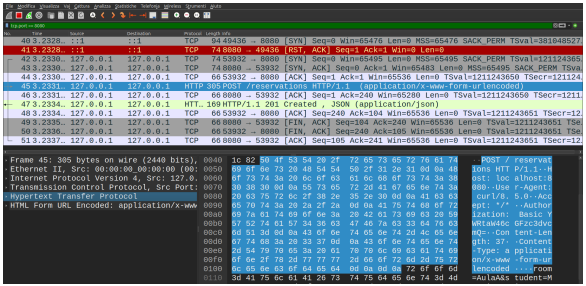
Metodo	Endpoint	Operazione nel Backend C	Success Code
`GET`	`/reservations`	Mutex Lock → Serializzazione JSON → Unlock	`200 OK`
`POST`	`/reservations`	Parsing URL-Encoded, Aggiunta struct in array	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca lineare ID e aggiornamento campi	`200 OK`
`DELETE`	`/reservations/{id}`	Memory shift a sinistra e ricompattazione array	`204 No Content`

16. Implementazione: Gestione degli Errori

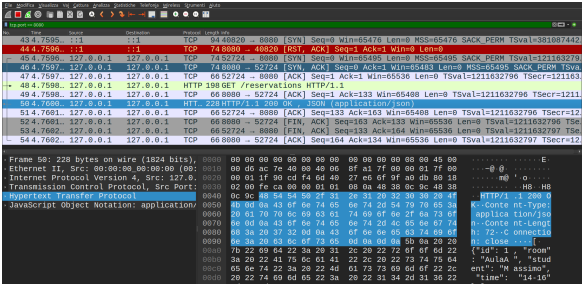
Il server gestisce le eccezioni e i casi limite restituendo codici di stato appropriati:

Categoria	Codice	Causa nel Backend
Client Error	400 `Bad Request`	Errore di parsing (es. parametri mancanti)
	401 `Unauthorized`	Token Base64 mancante o errato
	404 `Not Found`	L'ID richiesto non esiste nell'array
	413 `Payload Too Large`	Dimensione richiesta superiore al buffer
Server Error	500 `Internal Error`	Fallimento `malloc()` o errore interno
	503 `Unavailable`	Job Queue satura (Server sovraccarico)

17. Implementazione: Ottimizzazioni Architetturali



POST Request e Basic Auth



Risposta 200 OK (JSON)

14. Implementazione: Ottimizzazioni Architetturali

Sincronizzazione Centralizzata (Mutex)

L'architettura previene attivamente *race conditions* e corruzione dei dati in memoria. L'impiego rigoroso del **Mutex** (`pthread_mutex_t`) garantisce che l'accesso alla memoria condivisa avvenga in totale mutua esclusione per ogni Worker Thread, assicurando l'assoluta coerenza transazionale sia in lettura che in scrittura.

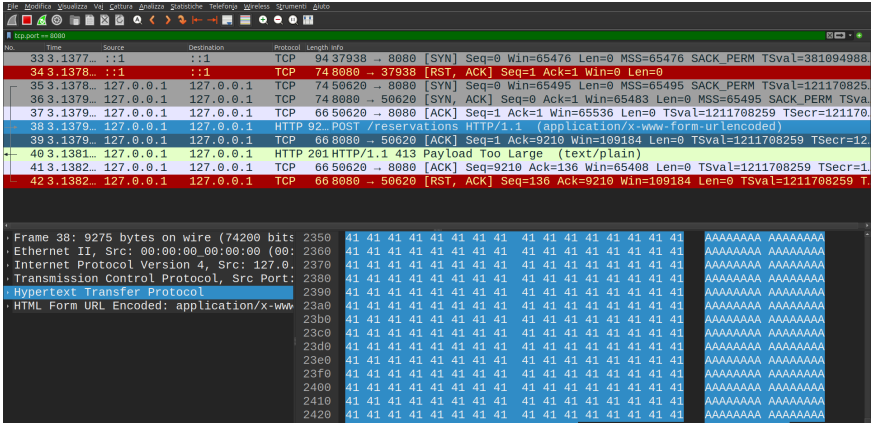
De-frammentazione della Memoria Dinamica

Poiché il C è sprovvisto di Garbage Collector, la rimozione logica di una prenotazione (`DELETE`) espone al rischio di frammentazione del vettore. L'implementazione impiega uno **shift a sinistra** progressivo tramite un **ciclo iterativo for** all'interno dell'area protetta, garantendo la ricompattazione continua dell'array dinamico senza librerie esterne.

18. Implementazione: Sicurezza

Mitigazione delle criticità intrinseche in ambiente C/POSIX:

- **Socket Leaks:** Dismissione sicura `close(fd)`.
- **Buffer Overflows:** Varianti sicure, errore 413.
- **Interruzione SIGPIPE:** Flag `MSG_NOSIGNAL`.



19. Qualità: Robustezza e Memory Management

Il progetto adotta rigorosi standard di sicurezza per prevenire vulnerabilità tipiche del linguaggio C, validati tramite l'uso di **Valgrind**.

Risultati Analisi Dinamica (Memcheck)

```
==16601== LEAK SUMMARY:
==16601==    definitely lost: 0 bytes in 0 blocks
==16601==    indirectly lost: 0 bytes in 0 blocks
==16601==    possibly lost: 2,720 bytes in 10 blocks
==16601==    still reachable: 40,064 bytes in 4 blocks
==16601== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0)
```

- **Definitely Lost: 0 bytes**: Conferma l'assenza di perdite di memoria nel cuore dell'implementazione.
- **Still Reachable**: Memoria ancora allocata dovuta all'interruzione forzata del server (`SIGINT`) prima del cleanup finale.
- **Sicurezza**: Utilizzo sistematico di `strncpy/snprintf` e gestione del segnale `SIGPIPE`.

20. Risultati: Load Testing e Performance

Il progetto è stato collaudato sfruttando lo strumento standard **Apache Bench (ab)**. La simulazione ha previsto: **10.000 richieste totali** con un livello di concorrenza di **100 client paralleli**.

Metriche di Prestazione

- **Total Requests:** 10.000
- **Concorrenza Esterna:** 100
- **Success Rate:** 100% (0 Errori)
- **Throughput:** $\sim$ 33.229 Req/Sec

Interpretazione Architetturale

L'assenza totale di connessioni droppate dimostra che la coda semaforica regge il carico e previene i fallimenti TCP. Inoltre, il tempo di risposta mediano di appena **3.0 ms** per richiesta conferma la leggerezza dei thread worker e la corretta implementazione dei lock Mutex sulle risorse condivise.

21. Conclusione e Sviluppi Futuri

L'implementazione in linguaggio C di un Thread Pool associato a socket BSD si è dimostrata una soluzione *memory-safe* ad altissime prestazioni per la costruzione di architetture HTTP concorrenti.

Sviluppi per Ambienti di Produzione:

1. **Persistenza dei Dati**: Passaggio dalla struttura in RAM ad un database relazionale leggero (come `SQLite3`) per mantenere lo stato dopo il riavvio del processo.
2. **I/O Asincrono (`epoll`)**: Sostituzione delle chiamate bloccanti `read()` con un loop *Event-Driven* nativo Linux per supportare il limite dei 10.000 Socket attivi.
3. **Cifratura**: Implementazione del protocollo *TLS/SSL* tramite librerie come *OpenSSL* per esporre API sicure (HTTPS).

Grazie per l'attenzione.